

UNIT-IV:

Front-End Web UI Frameworks and Tools:

⇒ Full stack web Development:

Full stack web development refers to the practice of working on both the client-side (front-end) & server-side (backend) of a web page.

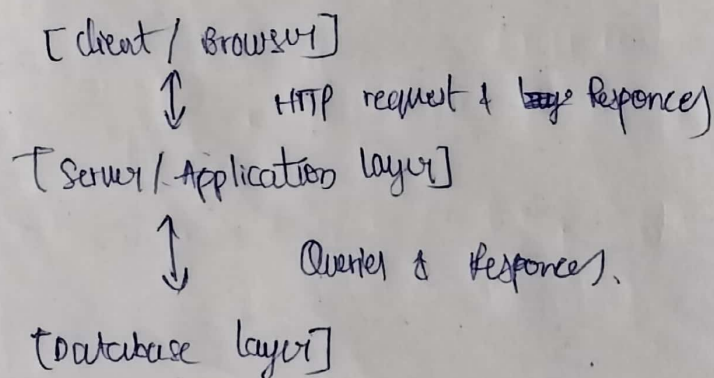
A full stack web developer is someone who can build a complete web application - from what a user sees in the browser to how data is stored and processed on the server.

→ Two Sides of the stack:

<u>layer</u>	<u>name</u>	<u>Responsibility</u>
Client side	front end	UI, UX, what users see & interact with
Server side	back end	Business logic, databases, APIs
Combined	full-stack	Both frontend - backend.

→ The three tier Architecture:

Most web applications follow 3 tier architecture



→ Front-End (Client Side)

The front-end is everything rendered in the browser.
Core technologies:

- HTML - Structure of the page
- CSS - Styling & layout
- JavaScript - Interactive & dynamic behaviour.

Front-end frameworks / libraries

<u>Tool</u>	<u>Type</u>	<u>Use</u>
React	Library	Component based UI
Angular	Framework	Full MVC front-end
Vue.js	Framework	Light weight, progressive UI
* Bootstrap	CSS framework	Responsive layouts & UI components

→ Backend (Server Side)

The backend handles logic, authentication, database communication and sends responses to the front end.

Core responsibilities:

- Handle HTTP requests (GET, POST, PUT, DELETE)
- Authenticate & authorize users.
- Interact with databases
- Return data (usually as JSON) to the front end.

Backend languages & Frameworks

<u>language</u>	<u>Framework</u>
Javascript	Node.js + Express
python	Django, flask
Java	spring Boot
PHP	Laravel
Ruby	Ruby on Rails

→ Databases

Database stores and manages the applications data.

<u>type</u>	<u>Examples</u>	<u>Best for</u>
Relational (SQL)	MySQL, PostgreSQL	Structured data, relationships
Non-Relationships (NoSQL)	MongoDB, Firebase	flexible, unstructured data

→ key concepts - CRUDs

Every database interaction maps to

<u>operation</u>	<u>SQL</u>	<u>HTTP method</u>
create	→ Insert	→ POST
Read	→ Select	→ GET
Update	→ UPDATE	→ PUT
delete	→ DELETE	→ DELETE

→ APIs - The Bridge Between Front-end & Back-end:

An API (Applications programming Interface) allows the front-end & backend to communicate.

REST API (most common):

1. front end sends an HTTP request to a URL endpoint
2. Backend processes it and returns a JSON response.

Front-end → GET /api/users → Back end → ({ id: 1, name: "Mani" })

Types of APIs:

REST → Stateless, HTTP-based, returns JSON.

GraphQL → Query-based, fetch only what you need

Web Socket → Real time, two way communication.

Fullstack flow (user logs into a web app)

1. user fills form (front-end - HTML/CSS/JS)
2. JS sends POST /api/login with credentials (API call)
3. Server receives request (Backend - Node.js / Express)
4. Server checks credentials in DB (Database - MongoDB)
5. Server returns just token (Response - JSON)
6. front-end stores token, redirect user (front-end logic)

⇒ Bootstrap:

What is Bootstrap:

Bootstrap is a free, open source CSS framework, developed by Twitter (now maintained by open source community). It provides ready-made components, a responsive grid system, and utility classes that help developers build consistent, mobile-friendly web interface faster.

In short: Bootstrap = pre-written CSS + JS components you plug into your HTML

Without Bootstrap

→ writing responsive layouts from scratch

→ Styling buttons, forms, navbars manually

→ Cross-browser inconsistencies

→ slow development

With Bootstrap

→ Built-in 12 column grid system.

→ pre-styled UI components

→ Normalized styles across browsers.

→ Ready to use classes.

Including Bootstrap:

Option 1 CDN (Quickest, no install)

```
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/
```

```
bootstrap@5.3.0/dist/css/bootstrap.min.css">
```

in head

before body

```
<script src="https://cdn.jsdelivr.net/npm/bootstrap@3.3.0/dist/js/bootstrap.bundle.min.js"></script>
```

option 2: npm (for project-based development)

npm install bootstrap.

// In your main js/entry file

import 'bootstrap/dist/css/bootstrap.min.css';

Grid System:

Bootstrap's grid is the most important concept. It is based on a 12-column layout using flexbox under the hood.

Structure:

```
<div class="Container">
```

```
<div class="row">
```

```
<div class="col-md-6"> left half </div>
```

```
<div class="col-md-6"> Right half </div>
```

```
</div>
```

```
</div>
```

Container → Centers content, adds padding.

Container fluid → full width container

row → Creates a horizontal group of columns.

col → Auto width column

col-xy → Column spanning x out of 12 units

Responsive Breakpoint:

<u>Breakpoint</u>	<u>Prefix</u>	<u>Screen Width</u>
Extra small	col-	< 576 px
Small	col-sm-	≥ 576 px
Medium	col-md-	≥ 768 px
Large	col-lg-	≥ 992 px
Extra-large	col-xl-	≥ 1200 px
xxl	col-xxl-	≥ 1400 px

eg: `<div class="row">`

`<div class="col-12 col-md-4">` Stacks on mobile
1/3 on desktop `</div>`

`<div class="col-12 col-md-4">` Stacks on +
1/3 on desktop `</div>`

`<div class="col-12 col-md-4">` 1/3 on desktop
1/3 on desktop `</div>`

`</div>`

On mobile → each div takes full width

On medium screen → each takes 1/3 (1/3)

(3) Card :

```
<div class="card" style="width: 18rem;">
```

```
<div class="card-body">
```

```
<h5 class="card-title">Card title </h5>
```

```
<p class="card-text">Some basic content </p>
```

```
<a class="btn btn-primary" href="#">
```

```
Go somewhere </a>
```

```
</div>
```

```
</div>
```

4) Form :

```
<form>
```

```
<div class="mb-3">
```

```
<label class="form-label">Email </label>
```

```
<input type="email" class="form-control"
```

```
placeholder="Enter Email" >
```

```
</div>
```

```
<button type="submit" class="btn btn-primary">
```

```
Submit </button>
```

```
</form>
```

5) Alert :

```
<div class="alert alert-success">Operation success </div>
```

```
<div class="alert alert-danger">Something went  
wrong </div>
```


utility classes

Bootstrap ships with dozens of utility classes for quick styling without writing custom CSS.

⇒ Spacing (margin & padding)

`m-{size}` → margin all sides

`mt-{size}` → margin top

`mb-{size}` → margin bottom

`p-{size}` → padding ~~left & right~~ all sides

`px-{size}` → padding left & right

`py-{size}` → padding top & bottom

Size: 0, 1, 2, 3, 4, 5, auto.

Text

class

`text-center`

`text-primary`

`fw-bold`

`fs-1` to `fs-6`

`text-uppercase`

Effect

Center align

blue color

Bold text

Font size

All caps

Display & Flexbox

`<div class="d-flex justify-content-between align-items-center"`

`</div>`

`d-none` → Hide element

`d-flex` → Enable flexbox

`d-none d-md-block`

Hide on mobile, Show on md+

`justify-content-center` → Center horizontally

`align-items-center` → center vertically

Core UI Components:

(1) Button

```
<button class="btn btn-primary">Primary </button>
```

class

color/style

btn-primary

blue

btn-success

green

btn-danger

red

btn-warning

yellow

btn-outline-*

Transparent with border

(2) Navbar

```
<nav class="navbar navbar-expand-lg navbar-dark  
bg-dark">
```

```
<div class="container">
```

```
<a class="navbar-brand" href="#">MyApp </a>
```

```
<div class="navbar-nav">
```

```
<a class="nav-link" href="#">Home </a>
```

```
<a class="nav-link" href="#">About </a>
```

```
</div>
```

```
</div>
```

```
</nav>
```

→ What is Git:

Git is a distributed version control system (VCS) that tracks changes in source code during software development. It allows multiple developers to collaborate, maintain history of changes, and revert to previous states when needed.

Repository (Repo) → A folder tracked by Git containing your project

Commit → A saved snapshot of changes

Branch → An independent line of development

Merge → Combining changes from two branches

Clone → Copying a remote repo to your local machine.

1. Setting up Git:

1. Install Git

<u>OS</u>	<u>Method</u>
windows	→ Download from git git-scm.com

macOS	→ brew install git or Xcode CI tools.
-------	--

Linux	→ sudo apt install git
-------	------------------------

(2) Configure Identity:

```
git config --global user.name "your name"
```

```
git config --global user.email "you@example.com"
```

This identity is attached to every commit you make.

(3) Verify Configuration:

```
git config --list
```

(4) Set default branch name (optional but recommended)

```
git config --global init.defaultBranch main
```

Initializing a Repository:

→ Start a New Repo Locally:

```
mkdir my-project
```

```
cd my-project
```

```
git init
```

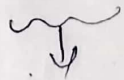
This creates a hidden .git folder - Git now tracks

this directory (check using `ls -a`)

hidden also shows

→ Clone an existing Remote Repo:

```
git clone <URL>
```



is on the github website.

Download the entire repo including its history

The Git workflow

Understanding the 3 stages of Git is fundamental

[Working directory]

⇒ files you're actively editing



git add

[Staging Area]

→ files marked / ready to be committed.



git commit

[Local repo]

→ committed snapshot
stored locally



git push

[Remote Repo]

→ Hosted repo on GitHub/
GitLab etc.

Basic Git commands

↳ Checking status & history.

git status # shows modified, staged, untracked files

git log # full commit history

git log --oneline # Compact commit history.

git diff # shows unstaged changes

git diff --staged # shows staged changes

→ staging & committing:

git add filename.txt → stage a specific file

git add . → stage all changes

git commit -m "your message" → commit with a msg

git commit -am "message" → stage staged files + commit in one step

→ undoing stages

git restore --filename.txt → discard unstaged changes in a file

git restore --staged filename → unstage a file (keep changes)

git ~~revert~~ ^{reverse} <commit hash> → Create new commit that undoes a commit

git reset --soft HEAD~1 → Undo last commit, keep changes staged

git reset --hard HEAD~1 → Undo last commit, discard all changes.

→ Branching

git branch → list all branches

git branch feature-login → Create new branch

git checkout feature-login → switch to branch

git checkout -b feature-login → create + switch in one step

git switch feature-login → modern alternative to checkout

git branch -d feature-login → Delete a branch

→ Online Git Repositories:

online platform host your Git repository remotely, enabling collaboration, backup and deployment.

Popular platforms:

- GitHub → open source, most popular, CI/CD ~~pipelines~~ integration
- GitLab → DevOps, built-in CI/CD pipeline
- Bitbucket → Atlassian ecosystem (Jira, Trello)

⇒ Connecting Local Repo to Remote (GitHub)

1. Create a repo on GitHub (no README)
2. Link your local repo to remote:

```
git remote add origin https://github.com/username---
```

3. push your code:

```
git push -u origin main
```

• 1-u sets upstream - after this, just use git push

4. verify remote:

```
git remote -v
```

→ Pushing, pulling & Fetching:

- git push → push local commits to remote
- git pull → ~~fetch~~ fetch + merge remote changes
- git fetch → Download remote changes (without merging)

⇒ .gitignore file

Tells Git which files / folders to never track.

.gitignore example

node-modules /
.env
*.log
dist /
.DS_Store

Always add node-modules /
and .env to .gitignore in
every project

Workflow :

1. git clone <repo> → Get the project.
2. git checkout -b feature -x → Create your feature branch
3. (make changes)
4. git add. && git commit -m "add feature x"
5. git push origin feature -x → push branch to remote
6. Open pull request (PR) on GitHub
7. Code review → Approved → Merge into main
8. git pull origin main → sync local with updated main

→ merging:

git checkout main

→ switch to target branch

git merge feature-login

→ Merge feature branch into main

Merge Conflict:

occurs when two branches modify the same line of a file

==== HEAD

your changes

=====

Incoming changes

>>>>>> feature-login

|| manually resolve → save the file → git add → git commit

→ stashing:

temporarily save work without committing.

git stash

stash current changes

git stash pop

Re-apply last stash

git stash list

See all stashes

git stash drop

Delete last stash

useful when you need to switch branches but aren't ready to commit.

Pros & Cons of Bootstrap:

- fast development
- Strong documentation
- Responsive out of the box
- Large community
- Consistent cross-browser
- sites can look "generic"
- adds unused CSS (bulky)
- overriding styles can be tricky
- learning curve for customize
- less design freedom vs Custom CSS.